

ESCUELA DE CIENCIAS EXACTAS E INGENIERÍA

PROGRAMA CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA
ARTIFICIAL

ASIGNATURA LENGUAJES DE PROGRAMACIÓN Y
TRANSDUCCIÓN

compilATOR

Diseño e Implementación de un Analizador Semántico

NASM x86-64

Estudiantes: Laura Vargas
Sebastián Tibata
Juan Riaño

Docente: William Frasser Acevedo

Fecha: Mayo 2026

Índice

1. Introducción	3
2. Objetivos	4
2.1. Objetivo General	4
2.2. Objetivos Específicos	4
3. Tecnologías de Software Utilizadas	5
3.1. NASM (Netwide Assembler)	5
3.2. GNU Linker (ld)	5
3.3. Equivalencia con Herramientas de Alto Nivel	5
3.4. IDE y Repositorio	6
3.5. Librerías Utilizadas	6
4. Diseño del Analizador Semántico	6
4.1. Reglas Semánticas del Lenguaje	6
4.2. Sistema de Tipos	7
4.3. Tabla de Símbolos: Diseño	7
4.4. Recorrido del AST	8
5. Integración entre el Parser y el Analizador Semántico	9
6. Implementación del Proyecto	10
6.1. Estructura de Archivos	10
6.2. Módulo <code>syntable.asm</code> — Tabla de Símbolos	11
6.3. Módulo <code>semantic.asm</code> — Walker del AST	13
6.3.1. Función <code>sem_walk</code>	13
6.3.2. Manejador de declaraciones	14
6.3.3. Manejador de comparaciones	15
6.4. CLI del binario <code>compi</code>	16
7. Proceso de Compilación	16
8. Pruebas Realizadas	17
8.1. Archivo de Prueba (<code>test.lator</code>)	17
8.2. Resultado Esperado — Entrada Válida	18
8.3. Pruebas de Error	18
8.3.1. Error: redeclaración de variable	18
8.3.2. Error: incompatibilidad de tipos en comparación	19
8.3.3. Error: flag <code>-s</code> omite el semántico	19

9. Manejo de Errores Semánticos	19
10.Dificultades Encontradas	20
10.1. Acceso a los offsets del AST desde <code>semantic.asm</code>	20
10.2. Distinción entre <code>NODE_ASSIGN</code> y <code>NODE_LET</code>	20
10.3. Identificadores externos no declarados	20
10.4. Inferencia de tipo para expresiones compuestas	20
11.Soluciones Implementadas	21
12.Resultados Obtenidos	21
13.Conclusiones	22
14.Recomendaciones	22
15.Bibliografía	23
A. Diseño de Pruebas	24
B. Resultados de Pruebas	24

1. Introducción

El análisis semántico constituye la tercera etapa del proceso de compilación. Recibe como entrada el Árbol de Sintaxis Abstracta (AST) construido por el analizador sintáctico y aplica sobre él un conjunto de verificaciones que la gramática formal no puede expresar: coherencia de tipos, ausencia de redeclaraciones y validez contextual de los operandos.

En este proyecto, el analizador semántico fue implementado desde cero en lenguaje ensamblador NASM x86-64, sin herramientas de soporte externas. Esta restricción obliga a gestionar explícitamente toda estructura de datos —tablas de símbolos, recorridos recursivos del AST, inferencia de tipos— que en lenguajes de alto nivel está encapsulada por la librería estándar o por generadores automáticos.

El lenguaje objetivo, LATOR, es declarativo y orientado a consultas. No existen bloques anidados ni ámbitos léxicos anidados: todo identificador declarado pertenece al ámbito global del programa. Esta restricción simplifica el análisis semántico respecto a un lenguaje imperativo de propósito general, pero no elimina la necesidad de inferir tipos ni de detectar redeclaraciones.

2. Objetivos

2.1. Objetivo General

Diseñar e implementar un analizador semántico en lenguaje ensamblador NASM x86-64 que recorra el AST producido por el analizador sintáctico, construya y consulte una tabla de símbolos con tipos inferidos, y detecte errores semánticos —redeclaraciones e incompatibilidades de tipo— reportando el número de línea correspondiente.

2.2. Objetivos Específicos

- Implementar una tabla de símbolos basada en una tabla hash de direccionamiento abierto con resolución de colisiones por sondeo lineal.
- Definir un sistema de tipos estático inferido, donde el tipo de cada variable declarada se deduce del nodo raíz de su expresión RHS.
- Implementar el recorrido recursivo del AST como un conjunto de manejadores por tipo de nodo (`NODE_ASSIGN`, `NODE_LET`, `NODE_CMP`, etc.).
- Detectar y reportar redeclaraciones de variables con el número de línea de la declaración original, terminando el proceso con código de salida 2.
- Detectar y reportar incompatibilidades de tipo en comparaciones cuando ambos operandos tienen tipos conocidos e incompatibles.
- Integrar el analizador semántico en el binario `compi`, reemplazando el antiguo binario `parser` con una CLI que soporte los flags `-s` y `-v`.

3. Tecnologías de Software Utilizadas

3.1. NASM (Netwide Assembler)

NASM es el ensamblador utilizado en todas las etapas del proyecto. Produce archivos objeto en formato ELF64 para Linux x86-64.

Características utilizadas en esta etapa:

- Directivas `global` y `extern` para la vinculación entre módulos (`symtable.asm` y `semantic.asm` exportan símbolos que `main.asm` y `parser.asm` importan).
- Directiva `%include` para incluir `ast.asm` dentro de `parser.asm` en tiempo de ensamblado.
- Constantes `equ` para los tipos de símbolo (`SYM_INT`, `SYM_FLOAT`, etc.) y los offsets del nodo AST.
- Secciones `.data`, `.bss` y `.text`.

3.2. GNU Linker (ld)

Vincula los archivos objeto generados por NASM en un binario ELF64 ejecutable sin dependencia de `libc`. Todas las operaciones de I/O se realizan mediante syscalls directas al kernel Linux.

3.3. Equivalencia con Herramientas de Alto Nivel

La guía de la asignatura describe el uso de herramientas como CUP y JFlex. La tabla siguiente extiende la correspondencia funcional al análisis semántico:

Concepto	Herramienta estándar	Este proyecto
Tabla de símbolos	HashMap de Java / estructura generada	<code>symtable.asm</code> : hash tabla estática, 256 buckets
Inferencia de tipos	Reglas semánticas en archivo <code>.cup</code>	<code>semantic.asm</code> : manejadores por <code>NODE_*</code>
Chequeo de tipos	Visitor pattern en Java	Recorrido recursivo con <code>sem_walk</code>
Reporte de errores	<code>report_error()</code> de CUP	<code>sem_error</code> : syscall <code>SYS_WRITE</code> a <code>stderr</code> + <code>SYS_EXIT 2</code>
Ámbito (scope)	Stack de tablas / <code>SymbolTable</code> anidada	Tabla plana global (lenguaje sin scopes)

Cuadro 1: Correspondencia funcional entre el enfoque estándar y la implementación en ensamblador.

3.4. IDE y Repositorio

El desarrollo se realizó en Sublime Text sobre Linux (Debian 13), con el repositorio gestionado mediante Git y alojado en GitHub:

<https://github.com/MrLaction/compiLATOR>

Las pruebas se ejecutan directamente en terminal con `make run`.

3.5. Librerías Utilizadas

No se utilizan librerías externas. El binario es completamente estático:

Syscall	Número	Uso
<code>SYS_READ</code>	0	Leer bytes del archivo fuente
<code>SYS_WRITE</code>	1	Imprimir mensajes a stdout / stderr
<code>SYS_OPEN</code>	2	Abrir el archivo <code>.lator</code>
<code>SYS_CLOSE</code>	3	Cerrar el descriptor de archivo
<code>SYS_EXIT</code>	60	Terminar el proceso

Cuadro 2: Syscalls utilizadas por el binario `compi`.

4. Diseño del Analizador Semántico

4.1. Reglas Semánticas del Lenguaje

El lenguaje LATOR impone tres categorías de restricciones semánticas que la gramática BNF no puede verificar:

1. **Unicidad de declaración:** un identificador no puede ser declarado más de una vez en el mismo programa.
2. **Coherencia de tipos en comparaciones:** si ambos operandos de una comparación tienen tipos conocidos e incompatibles (por ejemplo, `SYM_INT` vs. `SYM_STRING`), la comparación es semánticamente inválida.
3. **Identificadores externos:** los identificadores que aparecen como fuentes de datos (`users`, `orders`, `prices`, etc.) no están declarados en el programa. El analizador semántico los acepta silenciosamente, dado que LATOR es un lenguaje de consultas sobre colecciones externas.

4.2. Sistema de Tipos

El sistema de tipos es inferido y unidireccional: el tipo de una variable declarada se determina a partir del nodo raíz de su expresión RHS. No existe coerción implícita ni polimorfismo.

Constante	Valor	Condición de inferencia
<code>SYM_UNKNOWN</code>	0	RHS no tiene tipo estaticamente determinable
<code>SYM_INT</code>	1	RHS es <code>NODE_LIT_INT</code>
<code>SYM_FLOAT</code>	2	RHS es <code>NODE_LIT_FLOAT</code>
<code>SYM_BOOL</code>	3	RHS es <code>NODE_LIT_BOOL</code>
<code>SYM_STRING</code>	4	RHS es <code>NODE_LIT_STR</code>
<code>SYM_COLLECTION</code>	5	RHS es <code>NODE_FILTER</code> , <code>NODE_AGGR</code> o <code>NODE_RANGE</code>

Cuadro 3: Constantes de tipo inferido y condición de asignación.

Cuando el tipo no es directamente determinable desde el nodo raíz (por ejemplo, RHS es un `NODE_BINOP` o un `NODE_ID`), se asigna `SYM_UNKNOWN`. Esto es correcto: el lenguaje no exige que todo sea tipificable estáticamente.

4.3. Tabla de Símbolos: Diseño

La tabla de símbolos es una tabla hash de direccionamiento abierto con 256 buckets y sondeo lineal para la resolución de colisiones. Cada entrada ocupa 32 bytes:

Offset	Campo	Propósito
+0	<code>name_ptr</code>	Puntero al nombre internado en el string pool
+8	<code>sym_type</code>	Tipo inferido (<code>SYM_*</code>)
+16	<code>sym_line</code>	Número de línea de la declaración
+24	(reservado)	Padding para alineación a 32 bytes

Cuadro 4: Estructura de una entrada en la tabla de símbolos.

La función hash utilizada es **djb2 mod 256**:

```

1  sym_hash:                                     ; rdi = ptr to string -> rax
    = hash [0..255]
2      xor    eax, eax
3      mov    eax, 5381                          ; djb2 initial seed
4      xor    rcx, rcx
5  .hash_loop:
6      mov    cl, byte [rdi + rcx]
7      test   cl, cl
8      jz     .done
9      imul   eax, eax, 33                       ; hash * 33
10     add    eax, ecx                            ; + char
11     inc    rcx
12     jmp    .hash_loop
13 .done:
14     and    eax, 0xFF                          ; mod 256
15     ret

```

Listing 1: Función hash djb2 implementada en NASM.

4.4. Recorrido del AST

El analizador semántico implementa un recorrido recursivo del AST mediante la función `sem_walk(node)`, que despacha al manejador correspondiente según el campo `NODE_TYPE` del nodo:

Tipo de nodo	Acción del manejador
<code>NODE_PROGRAM</code>	Recorre la lista de sentencias recursivamente
<code>NODE_STMT_LIST</code>	Llama <code>sem_walk</code> sobre hijo izquierdo y derecho
<code>NODE_ASSIGN</code>	Infiere tipo del RHS; inserta nombre en tabla de símbolos
<code>NODE_LET</code>	Ídem <code>NODE_ASSIGN</code>
<code>NODE_CMP</code>	Infiere tipos de LHS y RHS; verifica compatibilidad
<code>NODE_FILTER</code>	Recorre expresión base y cláusula <code>where</code>
<code>NODE_AGGR</code>	Recorre fuente y filtro
<code>NODE_COND_AND</code> / <code>NODE_COND_OR</code>	Recorre ambos hijos
<code>NODE_COND_NOT</code>	Recorre hijo izquierdo
<code>NODE_IN_TEST</code>	Recorre expresión e ítems de la lista
<code>NODE_ID</code>	Acepta silenciosamente (fuente externa o ya declarada)
Nodos literales	Retornan su <code>SYM_*</code> correspondiente

Cuadro 5: Manejadores del recorrido semántico por tipo de nodo.

5. Integración entre el Parser y el Analizador Semántico

En el proyecto anterior, el binario `parser` realizaba únicamente el análisis sintáctico. En esta etapa, se introduce el binario `compi`, que integra lexer, parser y analizador semántico en un único ejecutable con una interfaz de línea de comandos.

Elemento	Implementación
Entrada	Archivo <code>.lator</code> pasado como argumento (<code>argv[1]</code>)
Lectura	<code>SYS_OPEN + SYS_READ</code> buffereado
Etapas	Lexer → Parser → Semántico (secuencial)
Flag <code>-s</code>	Detiene la ejecución después del parser; omite el semántico
Flag <code>-v</code>	Reservado para volcado del AST en etapas futuras
Códigos de salida	0 = OK, 1 = error sintáctico, 2 = error semántico

Cuadro 6: Integración de las tres etapas en el binario `compi`.

El flujo de ejecución es estrictamente secuencial: si el lexer emite `TK_ERROR`, el proceso termina antes de construir el AST. Si el parser falla, el semántico no se ejecuta. El semántico solo se invoca sobre un AST estructuralmente correcto.

```

1  _start:
2      ; parse arguments: file path, optional flags
3      call    parse_args          ; rax = fd, r12 = flags
4
5      ; lexer + parser: build AST
6      call    parse_program       ; rax = root node ptr
7
8      ; print "syntax OK" to stdout
9      mov     rdi, msg_syntax_ok
10     call    print_str
11
12     ; check -s flag: skip semantic if set
13     test    r12, FLAG_SYNTAX_ONLY
14     jnz     .exit_ok
15
16     ; semantic analysis
17     mov     rdi, rax             ; rax = root node from parser
18     call    sem_walk
19
20     ; print "semantic OK" to stdout
21     mov     rdi, msg_semantic_ok
22     call    print_str
23
24 .exit_ok:

```

```
25     mov    rax, SYS_EXIT
26     xor    rdi, rdi
27     syscall
```

Listing 2: Flujo de `_start` en `main.asm` del binario `compi`.

6. Implementación del Proyecto

6.1. Estructura de Archivos

```
1  compilATOR/
2  |- lexer/
3  |   |- symbol_table.asm  <- constantes de tokens, keywords
4  |   |- lexer.asm         <- DFA tokenizador, get_token
5  |   |- main.asm          <- entry point del binario lexer
6  |   standalone
7  |   |- Makefile
8  |   '-- test.src
9  |- parser/
10 |   |- ast.asm            <- arena allocator: 2048 nodos x 40
11 |   bytes
12 |   |- strpool.asm        <- string pool: 64 KB para lexemas
13 |   persistentes
14 |   |- parser.asm         <- parser LL(1) de descenso recursivo
15 |   |- symtable.asm       <- tabla hash de simbolos: 256 buckets
16 |   x 32 bytes
17 |   |- semantic.asm       <- walker del AST, inferencia de tipos,
18 |   chequeos
19 |   |- main.asm           <- entry point del binario compi (CLI)
20 |   |- grammar.md         <- gramatica BNF formal
21 |   |- Makefile
22 |   '-- test.lator
23 '-- README.md
```

Listing 3: Árbol de directorios del proyecto en la etapa 3.

Los módulos nuevos respecto a la etapa anterior son `symtable.asm` y `semantic.asm`. El módulo `main.asm` fue reescrito para implementar la CLI del binario `compi`.

6.2. Módulo `syntable.asm` — Tabla de Símbolos

Este módulo implementa la tabla hash de símbolos. Exporta tres funciones públicas:

- `sym_insert(name_ptr, type, line)`: inserta una nueva entrada. Si el nombre ya existe, emite el error de redeclaración y termina con código 2.
- `sym_lookup(name_ptr)`: devuelve un puntero a la entrada si el nombre existe, o NULL (cero) si no está registrado.
- `sym_hash(name_ptr)`: función hash interna (djb2 mod 256), exportada para uso en pruebas.

```

1  sym_insert:                                ; rdi = name_ptr, rsi = type, rdx
    = line
2      push rbp
3      mov  rbp, rsp
4      push r12
5      push r13
6      push r14
7
8      mov  r12, rdi                        ; save name_ptr
9      mov  r13, rsi                        ; save type
10     mov  r14, rdx                        ; save line
11
12     ; check for existing entry (redeclaration)
13     call sym_lookup                      ; rdi = name_ptr, rax = entry or
    0
14     test rax, rax
15     jnz  .redecl                         ; non-zero: already declared
16
17     ; compute bucket index
18     mov  rdi, r12
19     call sym_hash                        ; rax = bucket [0..255]
20     imul rax, rax, SYM_ENTRY_SIZE
21     lea  rcx, [sym_table]
22     add  rcx, rax                        ; rcx = first candidate bucket
23
24 .probe_loop:
25     cmp  qword [rcx + SYM_NAME], 0      ; empty slot?
26     je   .insert_here
27     add  rcx, SYM_ENTRY_SIZE            ; linear probe
28     lea  rax, [sym_table + 256 * SYM_ENTRY_SIZE]

```

```
29     cmp    rcx, rax
30     jl     .probe_loop
31     ; wrap around to beginning
32     lea    rcx, [sym_table]
33     jmp    .probe_loop
34
35 .insert_here:
36     mov    qword [rcx + SYM_NAME], r12
37     mov    qword [rcx + SYM_TYPE], r13
38     mov    qword [rcx + SYM_LINE], r14
39     xor    rax, rax
40     pop    r14
41     pop    r13
42     pop    r12
43     pop    rbp
44     ret
45
46 .redecl:
47     ; rax = pointer to existing entry
48     mov    r8, qword [rax + SYM_LINE] ; original declaration line
49     mov    rdi, r12                  ; name_ptr for error
50     ; message
51     mov    rsi, r8
52     call   sem_error_redecl          ; does not return (exit 2)
```

Listing 4: Implementación de sym_insert en symtable.asm.

6.3. Módulo semantic.asm — Walker del AST

Es el módulo central de esta etapa. Implementa el recorrido recursivo del AST y todas las verificaciones semánticas.

6.3.1. Función sem_walk

```

1  sem_walk:                                ; rdi = node_ptr -> rax =
    inferred SYM_* type
2      test rdi, rdi
3      jz     .null_node                    ; null node: return SYM_UNKNOWN
4
5      mov    rax, qword [rdi + NODE_TYPE]
6
7      cmp    rax, NODE_PROGRAM            ; dispatch table by node type
8      je     .handle_program
9      cmp    rax, NODE_STMT_LIST
10     je     .handle_stmt_list
11     cmp    rax, NODE_ASSIGN
12     je     .handle_decl
13     cmp    rax, NODE_LET
14     je     .handle_decl
15     cmp    rax, NODE_CMP
16     je     .handle_cmp
17     cmp    rax, NODE_LIT_INT
18     je     .ret_int
19     cmp    rax, NODE_LIT_FLOAT
20     je     .ret_float
21     cmp    rax, NODE_LIT_STR
22     je     .ret_string
23     cmp    rax, NODE_LIT_BOOL
24     je     .ret_bool
25     cmp    rax, NODE_FILTER
26     je     .handle_filter
27     cmp    rax, NODE_AGGR
28     je     .ret_collection
29     cmp    rax, NODE_RANGE
30     je     .ret_collection
31     ; default: recurse into children, return SYM_UNKNOWN
32     jmp     .handle_generic
33
34 .null_node:
```

```
35     mov    rax, SYM_UNKNOWN
36     ret
```

Listing 5: Dispatcher central de `sem_walk`.

6.3.2. Manejador de declaraciones

```
1  .handle_decl:
2      push rbp
3      mov  rbp, rsp
4      push r12
5      push r13
6
7      mov  r12, rdi                ; save node ptr
8
9      ; infer type from RHS (right child)
10     mov  rdi, qword [r12 + NODE_RIGHT]
11     call sem_walk                ; rax = inferred type
12     mov  r13, rax
13
14     ; retrieve name from left child's value field
15     mov  rdi, qword [r12 + NODE_LEFT]
16     mov  rdi, qword [rdi + NODE_VALUE] ; name_ptr
17
18     mov  rsi, r13                ; type
19     mov  rdx, qword [r12 + NODE_LINE] ; declaration line
20     call sym_insert              ; exits 2 on
21                                 redeclaration
22
23     mov  rax, r13
24     pop  r13
25     pop  r12
26     pop  rbp
27     ret
```

Listing 6: Manejador compartido para `NODE_ASSIGN` y `NODE_LET`.

6.3.3. Manejador de comparaciones

```

1  .handle_cmp:
2      push rbp
3      mov  rbp, rsp
4      push r12
5      push r13
6
7      mov  r12, rdi
8
9      ; infer LHS type
10     mov  rdi, qword [r12 + NODE_LEFT]
11     call sem_walk
12     mov  r13, rax                                ; lhs_type
13
14     ; infer RHS type
15     mov  rdi, qword [r12 + NODE_RIGHT]
16     call sem_walk                                ; rax = rhs_type
17
18     ; skip check if either side is SYM_UNKNOWN
19     cmp  r13, SYM_UNKNOWN
20     je   .cmp_ok
21     cmp  rax, SYM_UNKNOWN
22     je   .cmp_ok
23
24     ; types known: must be equal
25     cmp  r13, rax
26     je   .cmp_ok
27
28     ; type mismatch
29     mov  rdi, qword [r12 + NODE_LINE]
30     call sem_error_type_mismatch                ; exits 2
31
32 .cmp_ok:
33     mov  rax, SYM_BOOL                            ; comparison always
34         yields bool
35     pop  r13
36     pop  r12
37     pop  rbp
38     ret

```

Listing 7: Manejador de NODE_CMP: chequeo de compatibilidad de tipos.

6.4. CLI del binario compi

El binario `compi` reemplaza al antiguo `parser`. Acepta un archivo `.lator` como argumento posicional obligatorio y dos flags opcionales:

Invocación	Comportamiento	Código de salida
<code>./compi test.lator</code>	Lexer + Parser + Semántico	0 / 1 / 2
<code>./compi test.lator -s</code>	Lexer + Parser únicamente	0 / 1
<code>./compi test.lator -v</code>	Reservado (sin efecto aún)	0 / 1 / 2
<code>./compi</code>	Error de invocación	1
<code>./compi archivo.txt</code>	Error: extensión incorrecta	1

Cuadro 7: Modos de invocación del binario `compi`.

El binario verifica que el argumento termine en `.lator` antes de abrir el archivo. Si la extensión no coincide, imprime un mensaje de uso y termina con código 1.

7. Proceso de Compilación

```

1 # Ensamblar modulos del lexer
2 nasm -f elf64 ../lexer/symbol_table.asm -o symbol_table.o
3 nasm -f elf64 ../lexer/lexer.asm -o lexer.o
4
5 # Ensamblar modulos del parser y semantico
6 # parser.asm incluye ast.asm via %include en tiempo de
   ensamblado
7 nasm -f elf64 strpool.asm -o strpool.o
8 nasm -f elf64 symtable.asm -o symtable.o
9 nasm -f elf64 semantic.asm -o semantic.o
10 nasm -f elf64 parser.asm -o parser.o
11 nasm -f elf64 main.asm -o main.o
12
13 # Vincular en un solo binario
14 ld symbol_table.o lexer.o strpool.o symtable.o semantic.o parser
   .o main.o \
15 -o compi
16
17 # 0 simplemente:
18 make run < test.lator

```

Listing 8: Comandos de ensamblado y enlace del binario `compi`.

Los archivos generados y su función:

Archivo	Función
symbol_table.o	Datos estáticos: tokens, keywords, tabla de nombres
lexer.o	DFA tokenizador, función <code>get_token</code>
strpool.o	Pool de strings, función <code>intern_str</code>
symtable.o	Tabla hash de símbolos, <code>sym_insert</code> / <code>sym_lookup</code>
semantic.o	Walker del AST, inferencia de tipos, chequeos semánticos
parser.o	Parser + arena de nodos AST (ast.asm incluido)
main.o	Entry point <code>_start</code> , CLI <code>compi</code>
compi	Binario ELF64 ejecutable final

Cuadro 8: Archivos objeto y su responsabilidad en el binario final.

8. Pruebas Realizadas

8.1. Archivo de Prueba (`test.lator`)

El mismo archivo utilizado en la etapa 2 sirve como caso de prueba para el analizador semántico. Los ocho patrones del lenguaje cubren todos los tipos de declaración, agregación y filtro:

```

1  -- Filter users older than 18
2  result is users where age > 18
3
4  -- Filter with multiple conditions
5  active_users is users where age > 18 and active == true
6
7  -- Check ordering
8  sorted is numbers where every element <= next
9
10 -- Aggregate with condition
11 total is sum of prices where category == "food"
12
13 -- Find shortest path
14 let shortest_route be path from A to B where distance is min
15
16 -- Membership test with list
17 filtered is orders where user.region in ["north", "south"]
18
19 -- Arithmetic in filter
20 tax_total is sum of orders.amount where price + tax > 100.0
21

```

```
22 -- Find maximum
23 let best_score be results where score is max
```

Listing 9: Contenido de test.lator.

8.2. Resultado Esperado — Entrada Válida

```
1 $ make run
2 nasm -f elf64 ../lexer/symbol_table.asm -o symbol_table.o
3 nasm -f elf64 ../lexer/lexer.asm -o lexer.o
4 nasm -f elf64 strpool.asm -o strpool.o
5 nasm -f elf64 symtable.asm -o symtable.o
6 nasm -f elf64 semantic.asm -o semantic.o
7 nasm -f elf64 parser.asm -o parser.o
8 nasm -f elf64 main.asm -o main.o
9 ld symbol_table.o lexer.o strpool.o symtable.o semantic.o parser
  .o main.o \
10 -o compi
11 ./compi test.lator
12 syntax OK
13 semantic OK
```

Listing 10: Salida de compi sobre test.lator.

El binario acepta los ocho patrones sin errores y termina con código de salida 0.

8.3. Pruebas de Error

8.3.1. Error: redeclaración de variable

```
1 result is users where age > 18
2 result is prices where category == "food"
```

Resultado esperado:

```
1 syntax OK
2 semantic error: 'result' already declared on line 1
```

El analizador detecta que `result` fue declarado en la línea 1 e intenta redeclararse en la línea 2. Termina con código de salida 2.

8.3.2. Error: incompatibilidad de tipos en comparación

```
1 x is 42
2 y is "hello"
3 z is x where x == y
```

Resultado esperado:

```
1 syntax OK
2 semantic error: type mismatch in comparison on line 3
```

El analizador infiere `SYM_INT` para `x` y `SYM_STRING` para `y`. Al evaluar la comparación `x == y`, detecta que los tipos son incompatibles.

8.3.3. Error: flag `-s` omite el semántico

```
1 result is users where age > 18
2 result is prices where category == "food"
```

```
1 $ ./compi redecl.lator -s
2 syntax OK
```

Con el flag `-s`, el proceso termina tras el análisis sintáctico con código 0, sin ejecutar el analizador semántico.

9. Manejo de Errores Semánticos

El analizador detecta errores en dos situaciones:

1. **Redeclaración:** `sym_insert` verifica primero con `sym_lookup` si el nombre ya existe. Si la búsqueda retorna un puntero no nulo, llama a `sem_error_redecl` con el nombre y la línea original.
2. **Incompatibilidad de tipos:** `.handle_cmp` infiere los tipos de ambos operandos. Si ambos son distintos de `SYM_UNKNOWN` y entre sí, llama a `sem_error_type_mismatch` con el número de línea.

Ambas funciones de error imprimen el mensaje a `stderr` y terminan con `SYS_EXIT 2`:

Al igual que el parser, el analizador semántico no implementa recuperación de errores: al detectar el primer error, imprime el mensaje y termina. Esta es la estrategia correcta para un compilador de una pasada sobre un lenguaje que no admite scopes ni herencia de tipos.

Código	Significado	Etapas que lo emite
0	Éxito	Cualquier etapa
1	Error sintáctico o invocación incorrecta	Parser / <code>main.asm</code>
2	Error semántico	<code>semantic.asm</code> / <code>syntable.asm</code>

Cuadro 9: Códigos de salida del binario `compil`.

10. Dificultades Encontradas

10.1. Acceso a los offsets del AST desde `semantic.asm`

El módulo `semantic.asm` necesita acceder a los campos de los nodos AST (`NODE_TYPE`, `NODE_LEFT`, `NODE_RIGHT`, `NODE_VALUE`, `NODE_LINE`) que están definidos como constantes `equ` en `ast.asm`. Como `ast.asm` se incluye vía `%include` dentro de `parser.asm` y no como módulo independiente, las constantes no están disponibles automáticamente para `semantic.asm`.

La solución fue duplicar las definiciones de offset en `semantic.asm` con el comentario `; must match ast.asm`, análogo al problema de sincronización de constantes de tokens encontrado en la etapa 2.

10.2. Distinción entre `NODE_ASSIGN` y `NODE_LET`

Ambos tipos de nodo representan declaraciones y requieren el mismo tratamiento semántico. El primer diseño duplicaba el manejador. La solución fue fusionar ambos en un único `.handle_decl` al que el dispatcher salta para cualquiera de los dos tipos.

10.3. Identificadores externos no declarados

El lenguaje permite usar colecciones como `users`, `orders` o `prices` sin declararlas. El primer diseño emitía un error al encontrar un `NODE_ID` cuyo nombre no estaba en la tabla de símbolos. El comportamiento correcto es aceptarlos silenciosamente, dado que son referencias a fuentes externas.

La solución fue distinguir el contexto: `NODE_ID` que aparece como hijo izquierdo de `NODE_ASSIGN` o `NODE_LET` es el nombre que se está declarando (no se busca en la tabla); `NODE_ID` en cualquier otro contexto es una referencia que se acepta sin verificación.

10.4. Inferencia de tipo para expresiones compuestas

El nodo raíz de una expresión como `price + tax` es `NODE_BINOP`, no un nodo literal. El analizador semántico no propaga tipos a través de operaciones aritméticas en esta etapa — asigna `SYM_UNKNOWN`. Esto es deliberado: evitar una implementación incompleta de inferencia que generaría falsos positivos en el chequeo de tipos.

11. Soluciones Implementadas

Problema	Solución
Constantes de AST no visibles en semantic.asm	Se duplicaron las definiciones equ de offsets en <code>semantic.asm</code> con comentario <code>; must match ast.asm</code> .
Manejadores duplicados para NODE_ASSIGN / NODE_LET	Se fusionó en un único <code>.handle_decl</code> ; el dispatcher salta a él para ambos tipos de nodo.
Falsos positivos por identificadores externos	Se eliminó la verificación de <code>NODE_ID</code> en contextos de referencia; solo se verifica redeclaración al insertar.
Inferencia incompleta en NODE_BINOP	Se asigna <code>SYM_UNKNOWN</code> para nodos no literales; el chequeo de tipos solo actúa cuando <i>ambos</i> tipos son conocidos.
Sincronización de constantes SYM_* entre módulos	Se centralizaron en <code>symtable.asm</code> y se importan con <code>extern</code> desde <code>semantic.asm</code> .

Cuadro 10: Problemas encontrados y soluciones aplicadas.

12. Resultados Obtenidos

El analizador semántico acepta correctamente los ocho patrones del lenguaje y detecta los tres tipos de error implementados:

#	Sentencia	Resultado sintáctico
1	<code>result is users where age >18</code>	✓
2	<code>active_users is users where age >18 and active == true</code>	✓
3	<code>sorted is numbers where every element <= next</code>	✓
4	<code>total is sum of prices where category == "food"</code>	✓
5	<code>let shortest_route be path from A to B where distance is min</code>	✓
6	<code>filtered is orders where user.region in ["north", "south"]</code>	✓
7	<code>tax_total is sum of orders.amount where price + tax >100.0</code>	✓
8	<code>let best_score be results where score is max</code>	✓

Cuadro 11: Resultados de las pruebas de aceptación.

El binario produce `syntax OK` seguido de `semantic OK` y termina con código de salida 0 para entradas válidas.

Para entradas con redeclaración, produce `semantic error: 'nombre' already declared on line N` y termina con código 2.

Para entradas con incompatibilidad de tipos, produce `semantic error: type mismatch in comparison on line N` y termina con código 2.

13. Conclusiones

La implementación del analizador semántico en NASM x86-64 evidencia que las verificaciones semánticas —redeclaración, inferencia de tipos, compatibilidad en comparaciones— pueden implementarse directamente sobre el AST mediante un recorrido recursivo simple, sin estructuras de datos auxiliares más complejas que una tabla hash.

La restricción de ámbito plano del lenguaje LATOR simplifica considerablemente la tabla de símbolos respecto a un lenguaje con bloques: una tabla hash con sondeo lineal es suficiente, y no se requiere ninguna pila de ámbitos.

La decisión de asignar `SYM_UNKNOWN` a expresiones no literales es conservadora pero correcta: evita falsos positivos en el chequeo de tipos al precio de no detectar errores que requerirían inferencia más profunda. Esta compensación es aceptable en la etapa 3; la etapa 4 (generación de código intermedio) deberá refinar el sistema de tipos si es necesario.

14. Recomendaciones

- Centralizar todas las constantes de nodo AST en un archivo `ast_defs.asm` incluido por todos los módulos que accedan al AST, para eliminar la duplicación de offsets entre `parser.asm` y `semantic.asm`.
- Extender la inferencia de tipos a `NODE_BINOP`: si ambos operandos son del mismo tipo numérico, el resultado puede inferirse sin análisis adicional.
- Implementar la opción `-v` para volcar el AST anotado con tipos inferidos, facilitando la depuración de las etapas siguientes.
- Considerar recuperación de errores semánticos acumulativos: reportar todos los errores de redeclaración en una sola pasada en lugar de detenerse al primero.

15. Bibliografía

1. Aho, A., Lam, M., Sethi, R., Ullman, J. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
2. Intel Corporation. (2023). *Intel 64 and IA-32 Architectures Software Developer's Manual*.
3. System V Application Binary Interface — AMD64 Architecture Processor Supplement (2023).
4. NASM Development Team. (2023). *NASM — The Netwide Assembler* (versión 2.16). <https://www.nasm.us/doc/>
5. Repositorio del proyecto: <https://github.com/MrLaction/compilATOR>

A. Diseño de Pruebas

ID	Entrada	Propósito	Esperado
T01	result is users where age >18	Declaración simple	semantic OK
T02	active_users is users where age >18 and active == true	Condición compuesta	semantic OK
T03	sorted is numbers where every element <= next	Cláusula EVERY	semantic OK
T04	total is sum of prices where category == "food"	Agregación	semantic OK
T05	let shortest_route be path from A to B where distance is min	Range + IS EXTREME	semantic OK
T06	filtered is orders where user.region in ["north", "south"]	IN + lista	semantic OK
T07	tax_total is sum of orders.amount where price + tax >100.0	Arith en filtro	semantic OK
T08	let best_score be results where score is max	IS MAX	semantic OK
T09	Dos declaraciones de result	Redeclaración	error línea 1
T10	x is 42; y is "hi"; z is x where x == y	Incompatibilidad de tipos	error línea 3
T11	T09 con flag -s	Omisión del semántico	syntax OK, salida 0

Cuadro 12: Diseño de pruebas del analizador semántico.

B. Resultados de Pruebas

```

1 $ ./compi test.lator
2 syntax OK
3 semantic OK
4
5 $ ./compi redecl.lator
```

```
6 syntax OK
7 semantic error: 'result' already declared on line 1
8
9 $ ./compil type_mismatch.lator
10 syntax OK
11 semantic error: type mismatch in comparison on line 3
12
13 $ ./compil redecl.lator -s
14 syntax OK
```

Listing 11: Ejecución completa de la suite de pruebas.

Todas las pruebas producen el resultado esperado.